Question 1: For each question hereafter, give the only *correct* response. (1 mark)

- Which of the sentences below *cannot* be generated by the grammar on the right?

```
<S> -> a <S> c <B> | <A> | b
<A> -> c <A> | c
<B> -> d | <A>
```

  (A) `aabcccc`
  (B) `abcd`
  (C) `acccd`
  (D) `acdc`
  (E) `ccccc`

Answer: D

- The Backus-Naur Form (BNF) is a well-established language for defining:
  (A) Attribute grammars.
  (B) Context-free grammars.
  (C) Context-sensitive grammars.
  (D) Operational grammars.
  (E) Regular grammars.

Answer: B

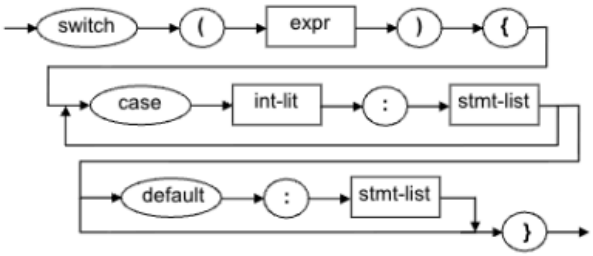
Question 2: **(a)** Write BNF rules to define simple *assignments* in C++, which may *cascade*. Valid examples include: `a=0; b=c; m=n=12; x=y=z;` (with the semi-colon). Assume the non-terminals `<const>` for numeric constants and `<var>` for variables are given. Is your grammar left-recursive? Explain. (2 marks)

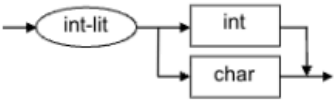
```
<assign> -> <var> = <term> ;
<term> -> <const> | <var> | <var> = <term>
```

With cascading, new terms are added to the right, hence the term rule must be right-recursive, and not left-recursive.

(b) Write EBNF rules for a *switch* statement in *C*. Note that it is *not* the same as C++; look up the definition if you need. Assume the following non-terminals are given: `<expr>` representing any valid C expression and `<stmt-list>` for a list of C statements. Draw the equivalent *syntax diagrams* as well. (3 marks)

```
<switch-stmt> -> switch '(' <expr> ')' '{'
                 { case <int-literal> : <stmt-list> }+      // 1 or more
                 [ default : <stmt-list> ]                  // optional
                 '}'
<int-literal> -> <int> | <char>                             // limitation of C (not C++)
```





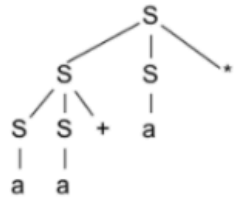
Question 3: **(a)** Indicate *what* the context-free grammar below (BNF) *generates,* and what `'a'` should be to make it a real programming language example. Give a leftmost *derivation* to show that the sentence `aa+a*` can be generated by this grammar, and draw the corresponding *parse tree.* (2 marks)

`<S> -> <S> <S> + | <S> <S> * | a` → this generates postfix expressions of **a**'s.

If **a** is a numeric variable or constant, this defines valid postfix expressions involving any mix of addition and multiplication. (e.g. in Forth, Postscript…) (cf. CMP 305 and 321 labs)

derivation:
```
<S> -> <S> <S> *
    -> <S> <S> + <S> *
    -> a <S> + <S> *
    -> a a + <S> *
    -> a a + a *
```

parse tree:



(b) Write a strict *leftmost reverse derivation* to show that the sentence `aa+aa*` cannot be generated by the grammar in part (a). Next, write a *mixed* reverse derivation, trying to go as far as possible… In both cases, indicate precisely how it stops and what the syntax error is. Explain briefly whether, in general, one approach is better than the other. (3 marks)

leftmost derivation:

```
->      <S>   <S> <S> *
->      <S>   <S>  a  *
not a
->      <S>    a   a  *
-> <S> <S> +   a   a  *
-> <S>  a  +   a   a  *
->  a   a  +   a   a  *
```

mismatch: two <S> must be followed by +|*

mixed derivation:

```
->      <S>        <S>
->      <S>   <S> <S>  *
->      <S>   <S>  a   *
->      <S>    a   a   *
-> <S> <S> +   a   a   *
-> <S>  a  +   a   a   *
->  a   a  +   a   a   *
```

error: missing op +|* after <S> <S>

There is no "better approach" because we cannot tell in advance which of a left-most derivation, a right-most derivation, or a mix, will yield the shortest proof. It depends on the grammar and what the particular syntax error is. However, only one algorithm can be implemented, and that is normally the leftmost derivation/recognition approach.

(c) Is the grammar in part (a) ambiguous? Answer first from principles, then using the sentence `aaa+*` as example to illustrate your explanations. (2 marks)

The rightmost operator must always be executed last; thus there is no ambiguity.

If parsing right to left, we can see that two <S> must always precede an operator, thus the only way to parse **aaa+*** is as **<S><S>*** where the first **<S>** must be **a** and the second **<S>** must be (recursively) **aa+**.

Question 4: **(a)** Consider the BNF grammar below (part of the English language). Draw *two parse trees* to show that the sentence "I saw an elephant in my pyjamas" has *two possible interpretations,* and that the grammar is ambiguous. Rephrase the sentence slightly, twice, to *make each interpretation clear.* (3 marks)

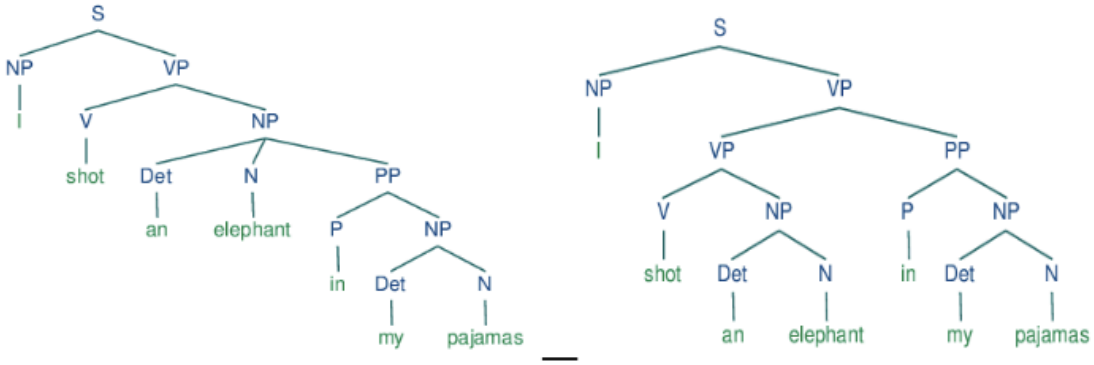
```
<S> -> <NP> <VP>
<PP> -> <P> <NP>
<NP> -> <D> <N> | <D> <N> <PP> | 'I'
<VP> -> <V> <NP> | <VP> <PP>
<D> -> 'a' | 'an' | 'my' | 'the' | 'your'
<N> -> 'banana' | 'elephant' | 'mouse' | 'pyjamas'
<V> -> 'found' | 'heard' | 'saw' | 'shot'
<P> -> 'in' | 'inside' | 'outside'
```

Proof: two parse trees (below) can be generated by this grammar, hence it is ambiguous.

The two interpretations, as clearly illustrated by the parse trees, are:

[1] "I saw an elephant which was in my pyjamas" / "I saw (an elephant in my pyjamas)"

[2] "I saw an elephant while I was in my pyjamas" / "I saw (an elephant) in my pyjamas"



(b) Show *using an example* why the grammar below *is* ambiguous. Rewrite the rules (minimally) so the grammar will be *unambiguous.* (3 marks)

```
<num> -> <bin> <num> | ε
<bin> -> 01 | <bin> 1 | 0 <bin> 1
```

The sentence 00111 *for instance has two derivations. The issue is that we can apply the rule* `<bin> -> <bin> 1` *either first or second to generate the extra* 1 *i.e.*

`bn -> b1n -> 0b11n -> 00111n` *vs.* `bn -> 0b1n -> 0b11n -> 00111n` *)*

correct:

```
<num> -> <bin> <num> | ε          // (same)
<bin> -> <aux> | 0 <bin> 1        // 2 levels – breaking the symmetry
<aux> -> <aux> 1 | 01
```

Question 5: The following rule set (on the left) defines a simple BNF grammar for Roman numerals of value less than 1000. We need to extend this grammar into an *Attribute Grammar,* as follows.

(a) Complete the *semantic rules* below (on the right) that specify how the `val` attribute is computed, that represents the decimal value of a Roman numeral. Note that `c=100`, `l=50`, `x=10`, `v=5`, `i=1`, and symbol values are added except: `i` before `v` or `x` indicates one less (e.g., `iv` is 4) and `x` before `l` or `c` indicates ten less (e.g., `xc` is 90). Use the semantic rules to *evaluate* the Roman numeral `xcviii`, showing clearly all the calculation steps. (3 marks)

```
<S> -> x<T><U>          <S>.val = <T>.val – 10 + <U>.val
<S> -> l<X>             <S>.val = <X>.val + 50
<S> -> <X>              <S>.val = <X>.val
<T> -> c                <T>.val = 100
<T> -> l                <T>.val = 50
<U> -> i<Y>             <U>.val = <Y>.val - 1
<U> -> v<I>             <U>.val = <I>.val + 5
<U> -> <I>              <U>.val = <I>.val
<X>[1] -> x<X>[2]       <X>[1].val = <X>[2].val + 10
<X> -> <U>              <X>.val = <U>.val
<Y> -> x                <Y>.val = 10
<Y> -> v                <Y>.val = 5
<I>[1] -> i<I>[2]       <I>[1].val = <I>[2].val + 1
<I> -> ε                <I>.val = 0
```

```
'xcviii'.val = 'c'.val – 10 + 'viii'.val
             = 100 – 10 + 'iii'.val + 5
             = 95 + 'ii'.val + 1
             = 96 + 'i'.val + 1
             = 97 + ''.val + 1
             = 98
```

(b) The above grammar allows illegal forms such as `xiiii` or `xxxxx`. Add the necessary *attribute/s, semantic rule/s, and predicate/s* to ensure that generated sentences contain groups of at most 3 `i` or 3 `x`. Draw a *synthetized, decorated parse tree* showing that `xcviiii` is *not* a valid Roman numeral. (4 marks)

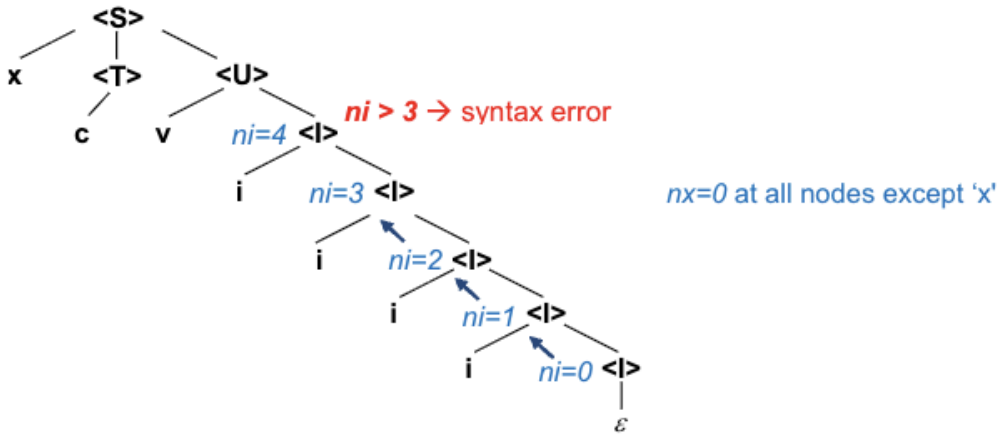
attributes: `nx` and `ni` : number of `x` and `i` chars, respectively

predicates: `<X>.nx <= 3` and `<I>.ni <= 3`

BNF rules:	semantic rules:
`<S> -> x<T><U>`	`<S>.nx = 1 + <T>.nx + <U>.nx ,` `<S>.ni = <T>.ni + <U>.ni`
`<S> -> l<X>`	`<S>.nx = <X>.nx , <S>.ni = <X>.ni`
`<S> -> <X>`	`<S>.nx = <X>.nx , <S>.ni = <X>.ni`
`<T> -> c`	`<T>.nx = <T>.ni = 0`
`<T> -> l`	`<T>.nx = <T>.ni = 0`
`<U> -> i<Y>`	`<U>.nx = <Y>.nx , <U>.ni = 1 + <Y>.ni`
`<U> -> v<I>`	`<U>.nx = <I>.nx , <U>.ni = <I>.ni`
`<U> -> <I>`	`<U>.nx = <I>.nx , <U>.ni = <I>.ni`
`<X>[1] -> x<X>[2]`	`<X>[1].nx = 1 + <X>[2].nx ,` `<X>[1].ni = <X>[2].ni`
`<X> -> <U>`	`<X>.nx = <U>.nx , <X>.ni = <U>.ni`
`<Y> -> x`	`<Y>.nx = 1 , <Y>.ni = 0`
`<Y> -> v`	`<Y>.nx = 0 , <Y>.ni = 0`
`<I>[1] -> i<I>[2]`	`<I>.nx = 0 , <I>[1].ni = 1 + <I>[2].ni`
`<I> -> ε`	`<I>.nx = 0 , <I>.ni = 0`

decorated parse tree:

derivation: `<S> -> x<T><U> -> xc<U> -> xcv<I> -> xcvi<I> -> xcvii<I> -> xcviii<I> -> xcviiii<I> -> xcviiii`



nx=0 at all nodes except 'x'

(Note: The semantic rules of an Attribute Grammar specify how to update attribute values e.g., in this case, the roman numeral value, number of `x` *and* `i` *characters, respectively. This is different from Denotational Semantics, where the semantic rules explain how language instructions work e.g., the addition of two numbers, assignments, 'for' loops, 'if' statements, etc. and not only how to derive the value of a number. Contrarily, DS is not concerned with syntax, hence would know nothing about the number of* `x` *and* `i` *…)*

Question 6: *Denotational semantics* can be used to justify ways of reasoning about programs. To take a simple example, we can *prove* that setting a variable to 1, then adding 2 to it, is equivalent to setting the variable to 3 in the first place. Formally, the following equivalence holds for every program state `s` (i.e., both expressions have the same meaning):

$$M_e(\text{" a=1; a=a+2; ", s}) == M_e(\text{" a=3; ", s})$$

Give a detailed proof, using the semantic functions given in Sebesta's textbook i.e., M_{dec} for decimal numbers, M_a for assignments, M_e for expressions, and showing the state `s` of the program at every step. (4 marks)

Recall the state of the program is characterised by the current value of all variables:

$s = \{\langle i_1, v_1\rangle, \langle i_2, v_2\rangle, \ldots, \langle i_n, v_n\rangle\}$

First, $M_e(\text{"a=1", s}) = M_a(x := E, s)$ where x is **a** and expression E is **1**.

This results in a state **s'** where all variables remain unchanged except **a** which now takes the value $M_e(E, s') = M_{dec}(1, s') = 1$.

Next, $M_e(\text{"a=a+2", s'}) = M_a(x := E, s')$ where x is **a** and expression E is **a+2**.

This is a binary expression, where both left and right operands are defined, hence
$M_e(E, s'') = M_e(a, s'') + M_e(2, s'') = \mathbf{Varmap}(a, s'') + M_{dec}(2, s'') = 1 + 2 = 3$.

This results in a new state **s''** where all variables remain unchanged except **a** which now has a value of **3**. (That is the result of executing the "program".)

Alternatively, $M_e(\text{"a=3", s}) = M_a(x := E, s)$ where x is **a** and expression E is **3**.

This results in a new state **s'** where all variables retain their value except **a** which has now the value $M_e(E, s) = M_{dec}(3, s) = 3$. (That is the result of this "program".)

Hence the state of the program is the same in both cases. *[QED]*